Black Box API Testing, Q3 Report

March 23, 2026

1 Objective

This part of the assignment performs black-box testing on the QuickCart REST API using only the given API documentation. The internal implementation was not available. The testing focused on request validation, response structure, status codes, returned data, and user-visible business rules.

2 Approach

Two kinds of testing were used.

First, manual exploratory testing was used to understand endpoint behavior from the documentation, inspect seeded data through the admin endpoints, and identify likely edge cases.

Second, automated tests were written using only `pytest` and `requests`. These tests check:

- correct HTTP status codes,
- JSON response structure,
- correctness of values returned by the API,
- valid requests,
- invalid inputs,
- missing fields,
- wrong data types,
- boundary values.

The automated suite targets the API at `http://127.0.0.1:8080/api/v1`. Admin requests use the `X-Roll-Number` header, and user-scoped requests use both `X-Roll-Number` and `X-User-ID`. I had initially used 123456 as the header placeholder, but The final submission uses the my roll number, `2024111034`, in the test configuration and documented request examples.

3 Automated Execution Summary

The final automated run for the submitted black-box suite produced the following result:

- Total automated test cases: 44
- Passed: 36
- Failed: 8

The failing tests correspond to real API bugs, not test setup issues.

4 Test Case Design

Table 1 lists the designed black-box test cases. The "Actual Result" column records the observed behavior during execution.

Table 1: Black-box test cases for QuickCart API

ID	Input / Endpoint	Expected Output	Justification	Actual Result
BB01	`GET /admin/users` with valid roll header	`200` with JSON list containing `user_id`	Confirms admin inspection endpoint works and returns structured data	Matched expected
BB02	`GET /admin/users` without `X-Roll-Number`	`401`	Missing required header must be rejected	Matched expected
BB03	`GET /admin/users` with non-integer roll header	`400`	Checks invalid header type handling	Matched expected
BB04	`GET /profile` without `X-User-ID`	`400`	User endpoints must reject missing user header	Matched expected
BB05	`GET /profile` with invalid `X-User-ID` type	`400`	Checks wrong data type in required header	Matched expected
BB06	`GET /profile` with valid headers	`200` and profile JSON fields present	Basic valid request test for user profile retrieval	Matched expected
BB07	`PUT /profile` with name of length 1	`400`	Boundary check for minimum valid name length	Matched expected
BB08	`PUT /profile` missing phone field	`400`	Required-field validation must work	Matched expected
BB09	`PUT /profile` with 5-digit phone number	`400`	Boundary check for exact 10-digit phone rule	Matched expected

ID	Input / Endpoint	Expected Output	Justification	Actual Result
BB10	`GET /products` list vs admin inactive products	Inactive products should not appear	Confirms filtering rule from documentation	Matched expected
BB11	`GET /products/999999`	`404`	Missing resource should be handled cleanly	Matched expected
BB12	`GET /products?search=Apple`	`200`; returned names should match search	Confirms search behavior	Matched expected
BB13	`GET /products?sort=price_asc` and `price_desc`	Returned products should be sorted correctly	Confirms documented sorting support	Matched expected
BB14	`POST /addresses` with invalid label	`400`	Invalid enum-like input should be rejected	Matched expected
BB15	`POST /addresses` missing street	`400`	Required-field validation for address creation	Matched expected
BB16	`POST /addresses` with valid 6-digit pincode	`200` and created address object	Valid positive case for address creation	Failed: API returned `400 Invalid pincode`
BB17	`POST /cart/add` with quantity 0	`400`	Boundary test for minimum quantity 1	Failed: API returned `200`
BB18	`POST /cart/add` with string quantity	`400`	Wrong data type validation for cart quantity	Matched expected
BB19	`POST /cart/add` with missing product id	`404`	Missing resource handling	Matched expected
BB20	Add same product twice to cart	Quantity should accumulate to 2	Confirms documented repeated-add behavior	Matched expected
BB21	Get cart after repeated add of same product	Item subtotal should equal quantity times unit price	Checks cart item price calculation	Failed: subtotal incorrect
BB22	Get cart after adding two different products	Cart total should equal sum of item subtotals	Checks cart total calculation and counting of all items	Failed: total incorrect
BB23	`POST /coupon/apply` with expired coupon	`400`	Confirms coupon expiry validation	Matched expected
BB24	`POST /wallet/add` with string amount	`400`	Wrong data type validation for wallet amount	Matched expected

ID	Input / Endpoint	Expected Output	Justification	Actual Result
BB25	`POST /wallet/add` with amount above 100000	`400`	Boundary check for wallet top-up maximum	Matched expected
BB26	`POST /wallet/pay` with amount 1	Wallet balance should reduce by exactly 1	Confirms exact money deduction logic	Failed: balance reduced by more than 1
BB27	`POST /loyalty/redeem` with 0 points	`400`	Boundary test for minimum redeemable points	Matched expected
BB28	`POST /checkout` with payment method `UPI`	`400`	Invalid payment method should be rejected	Matched expected
BB29	`POST /checkout` with empty cart	`400`	Empty cart checkout must not be allowed	Failed: API returned `200`
BB30	`GET /orders/999999`	`404`	Missing order lookup should fail cleanly	Matched expected
BB31	`POST /orders/999999/cancel`	`404`	Missing order cancellation should fail cleanly	Matched expected
BB32	`POST /products/{id}/reviews` with rating 6	`400`	Boundary test for maximum rating 5	Matched expected
BB33	Add a new review and then fetch reviews	Average rating should be a proper decimal value	Checks data correctness for computed average	Failed: average returned as truncated integer
BB34	`POST /support/ticket` with subject shorter than 5 chars	`400`	Boundary validation for subject length	Matched expected
BB35	`POST /support/ticket` missing message	`400`	Required-field validation for ticket creation	Matched expected
BB36	`POST /support/ticket` with valid body	`200`; new ticket status should be OPEN	Valid request for support workflow start	Matched expected
BB37	Create ticket, move OPEN to IN_PROGRESS, then try IN_PROGRESS to OPEN	Final update should return `400`	Confirms one-way status workflow rule	Failed: API returned `200`
BB38	`GET /admin/products`	JSON list containing `product_id`	Verifies admin product inspection structure	Matched expected

ID	Input / Endpoint	Expected Output	Justification	Actual Result
BB39	`GET /admin/orders`	JSON list containing `order_id`	Verifies admin order inspection structure	Matched expected
BB40	`GET /admin/carts`	JSON list containing `cart_id`	Useful for validating global cart state	Matched expected
BB41	`GET /admin/coupons`	JSON list containing `coupon_code`	Useful for selecting real coupons during tests	Matched expected
BB42	`GET /admin/tickets`	JSON list containing `ticket_id`	Useful for support-ticket verification	Matched expected
BB43	`GET /admin/addresses`	JSON list containing `address_id`	Useful for address-state verification	Matched expected
BB44	All tested user endpoints	Responses should be JSON with expected structure	Ensures automated checks validate response shapes, not just status codes	Matched expected across passing cases

5 Why Both Manual and Automated Testing Were Used

In black-box testing, test cases are first designed from the API specification without using internal code. Manual testing helps understand how the API behaves in practice, discover seeded data values, and identify useful edge cases. Automated testing then makes those checks repeatable and objective. For this assignment, both were useful:

- manual testing helped understand the endpoints and data,
- automated pytest cases produced repeatable evidence,
- the final report is based on the automated results and the observed API behavior.

6 Bug Report

The following bugs were reproduced by the automated test suite.

6.1 Bug 1: Valid 6-digit pincode is rejected

Endpoint tested: `POST /api/v1/addresses`

Request payload:

- Method: `POST`

- URL: `http://127.0.0.1:8080/api/v1/addresses`
- Headers: `X-Roll-Number: 2024111034`, `X-User-ID: 1`
- Body: `{"label":"OTHER","street":"12345 Test Street <id>","city":"Delhi","pincode"`

Expected result: Address should be created successfully with `200`.
Actual result: API returned `400` with `{"error":"Invalid pincode"}`.

6.2 Bug 2: Cart accepts quantity 0

Endpoint tested: `POST /api/v1/cart/add`
Request payload:

- Method: `POST`
- URL: `http://127.0.0.1:8080/api/v1/cart/add`
- Headers: `X-Roll-Number: 2024111034`, `X-User-ID: 1`
- Body: `{"product_id":1,"quantity":0}`

Expected result: API should reject quantity 0 with `400`.
Actual result: API returned `200` and added the item.

6.3 Bug 3: Cart item subtotal is calculated incorrectly

Endpoint tested: `GET /api/v1/cart` after adding the same product twice
Request payload:

- Method: `GET`
- URL: `http://127.0.0.1:8080/api/v1/cart`
- Headers: `X-Roll-Number: 2024111034`, `X-User-ID: 1`
- Body: none

Expected result: For quantity 2 and unit price 120, subtotal should be 240.
Actual result: API returned subtotal `-16`.

6.4 Bug 4: Cart total is calculated incorrectly

Endpoint tested: `GET /api/v1/cart` after adding two different products
Request payload:

- Method: `GET`
- URL: `http://127.0.0.1:8080/api/v1/cart`
- Headers: `X-Roll-Number: 2024111034`, `X-User-ID: 1`
- Body: none

Expected result: Cart total should equal the sum of all item subtotals.
Actual result: API returned total `120` while item subtotals summed to `-6`.

6.5 Bug 5: Wallet payment deducts more than requested

Endpoint tested: `POST /api/v1/wallet/pay`
Request payload:

- Method: `POST`
- URL: `http://127.0.0.1:8080/api/v1/wallet/pay`
- Headers: `X-Roll-Number: 2024111034`, `X-User-ID: 1`
- Body: `{"amount":1}`

Expected result: Wallet balance should decrease by exactly 1.
Actual result: The observed balance dropped by more than 1, showing incorrect deduction logic.

6.6 Bug 6: Checkout allows an empty cart

Endpoint tested: `POST /api/v1/checkout`
Request payload:

- Method: `POST`
- URL: `http://127.0.0.1:8080/api/v1/checkout`
- Headers: `X-Roll-Number: 2024111034`, `X-User-ID: 1`
- Body: `{"payment_method":"COD"}`

Expected result: API should reject empty-cart checkout with `400`.
Actual result: API returned `200` and created an order.

6.7 Bug 7: Review average is not returned as a proper decimal

Endpoint tested: `GET /api/v1/products/{id}/reviews` after adding a new review
Request payload:

- Method: `GET`
- URL: `http://127.0.0.1:8080/api/v1/products/1/reviews` (product id varies by test data)
- Headers: `X-Roll-Number: 2024111034`, `X-User-ID: 1`
- Body: none

Expected result: Average rating should be the true decimal average.
Actual result: API returned a truncated integer value such as `3` instead of the correct decimal average.

6.8 Bug 8: Support ticket workflow allows backward status transition

Endpoint tested: `PUT /api/v1/support/tickets/{ticket_id}`
Request payload:

- Method: `PUT`
- URL: `http://127.0.0.1:8080/api/v1/support/tickets/<ticket_id>`
- Headers: `X-Roll-Number: 2024111034`, `X-User-ID: 1`
- Body: `{"status":"OPEN"}` after the ticket is already in `IN_PROGRESS`

Expected result: API should reject the backward transition with `400`.
Actual result: API returned `200` and allowed the status change.